

Parallelism Taxonomy for DeepSeek

Deep Dive into DeepSeek — Chapter 25

Yin Yang

Foundation Books Project

The one rule of this chapter

A parallelism plan is an **ownership account**, not a list of degrees.

- Every logical object — an example, a tensor shard, a layer, a routed expert row, a sequence chunk, an optimizer moment, a request, a cache block — needs a named owner, a named consumer, and a communication path between them.
- Writing $D_{tp} = 2$ says nothing by itself. What matters is which object that degree partitions, and which collective restores it.
- Training and serving reuse the same six-letter vocabulary but assign it to different objects entirely.

Goal: follow one eight-rank teaching plan through data, tensor, pipeline, expert, and sequence/context ownership, then watch one routed token cross DeepSeek's released component contracts end to end.

1. Ownership vocabulary

Roadmap

1. Ownership vocabulary
2. Training axes: data, tensor, pipeline

1. Ownership vocabulary
2. Training axes: data, tensor, pipeline
3. MoE axes: expert parallelism and placement

1. Ownership vocabulary
2. Training axes: data, tensor, pipeline
3. MoE axes: expert parallelism and placement
4. Sequence axis and composition

1. Ownership vocabulary
2. Training axes: data, tensor, pipeline
3. MoE axes: expert parallelism and placement
4. Sequence axis and composition
5. Training vs. serving: same vocabulary, different owners

Ownership vocabulary

Rank, group, shard, replica

Term	Meaning
Device, worker, rank	compute target; participating process; that process's index inside a communication group
Parallel group	an ordered set of ranks that jointly own one partition dimension or one collective family
Shard vs. replica	a proper subset of an object plus its reconstruction rule, vs. a complete copy on more than one rank

One rank can sit in several named groups at once. Correctness means using the right ordered group for each collective — not just having enough ranks.

The eight-rank teaching plan

$$\Gamma = (D_{\text{dp}} = 2, D_{\text{tp}} = 2, D_{\text{pp}} = 2, D_{\text{ep}} = 4, D_{\text{sp}} = 1, Z = \text{sharded})$$

- Eight accelerators, two nodes. $B = 64$ sequences, $L = 8$ blocks, $E = 16$ routed experts, top-2 routing.

The eight-rank teaching plan

$$\Gamma = (D_{dp} = 2, D_{tp} = 2, D_{pp} = 2, D_{ep} = 4, D_{sp} = 1, Z = \text{sharded})$$

- Eight accelerators, two nodes. $B = 64$ sequences, $L = 8$ blocks, $E = 16$ routed experts, top-2 routing.
- The degrees do **not** multiply into disjoint rank pools — $dp=2, tp=2, pp=2$ are independent coordinates whose product is the eight-rank world; $ep=4$ and $sp=1$ are overlapping group families layered on top.

The eight-rank teaching plan

$$\Gamma = (D_{dp} = 2, D_{tp} = 2, D_{pp} = 2, D_{ep} = 4, D_{sp} = 1, Z = \text{sharded})$$

- Eight accelerators, two nodes. $B = 64$ sequences, $L = 8$ blocks, $E = 16$ routed experts, top-2 routing.
- The degrees do **not** multiply into disjoint rank pools — $dp=2, tp=2, pp=2$ are independent coordinates whose product is the eight-rank world; $ep=4$ and $sp=1$ are overlapping group families layered on top.

This is a teaching fixture, not a reconstruction of DeepSeek-V3's reported topology. [DERIVED:
chapter running example]

**Training axes: data, tensor,
pipeline**

Data parallelism and ZeRO: two different ownership moves

Move	What it partitions
Data parallelism	examples/tokens; computation stays replicated
ZeRO-style sharding	parameters, gradients, optimizer moments

$g(\theta) = \frac{1}{D_{\text{dp}}} \sum_i g_i(\theta)$: with $D_{\text{dp}} = 2$ and local gradients 0.30, 0.10, the synchronized gradient is 0.20 — neither replica may update from its own gradient alone.

Sharding does not delete the 96 GB toy optimizer ledger; it moves half of it behind a gather/reduce/scatter boundary at update time.

Tensor parallelism: column shards concatenate, row shards reduce

For $Y = XW$, a column shard $W = [W_0 \ W_1]$ gives $Y_0 = XW_0$, $Y_1 = XW_1$, and $Y = [Y_0 \ Y_1]$ — concatenation, no reduction required.

Tensor parallelism: column shards concatenate, row shards reduce

For $Y = XW$, a column shard $W = [W_0 \ W_1]$ gives $Y_0 = XW_0$, $Y_1 = XW_1$, and $Y = [Y_0 \ Y_1]$ — concatenation, no reduction required.

A row shard splits the reduction dimension: $Y = X_0 W_0 + X_1 W_1$. If the two partial sums are not summed over the tensor group, the model has computed two *incomplete* projections, not one sharded one.

Tensor parallelism: column shards concatenate, row shards reduce

For $Y = XW$, a column shard $W = [W_0 \ W_1]$ gives $Y_0 = XW_0$, $Y_1 = XW_1$, and $Y = [Y_0 \ Y_1]$ — concatenation, no reduction required.

A row shard splits the reduction dimension: $Y = X_0 W_0 + X_1 W_1$. If the two partial sums are not summed over the tensor group, the model has computed two *incomplete* projections, not one sharded one.

The tensor collective — concatenate or reduce — is part of the layer's contract, not an optional optimization.

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:



Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:

	1	2	3	4	5	6	7	8	9
s_0	F	F	F	F	F	F			
s_1		F	F	F	F	F	F		

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:

	1	2	3	4	5	6	7	8	9
s_0	F	F	F	F	F	F			
s_1		F	F	F	F	F	F		
s_2			F	F	F	F	F	F	

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:

	1	2	3	4	5	6	7	8	9
s_0	F	F	F	F	F	F			
s_1		F	F	F	F	F	F		
s_2			F	F	F	F	F	F	
s_3				F	F	F	F	F	F

stage 3: idle slots 1–3, warmup

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:

	1	2	3	4	5	6	7	8	9
s_0	F	F	F	F	F	F			
s_1		F	F	F	F	F	F		
s_2			F	F	F	F	F	F	
s_3				F	F	F	F	F	F

stage 3: idle slots 1–3, warmup

At least $D_{pp} - 1 = 3$ warmup slots are idle on the last stage before steady state — and a matching drain effect appears at the end.

Pipeline parallelism: stages own layers, bubbles are named idle time

$D_{pp} = 2$ splits $L = 8$ blocks into stage 0 (blocks 1–4) and stage 1 (blocks 5–8); the activation boundary between them is a point-to-point transfer, not a collective.

With four stages and six microbatches, one forward interval per microbatch per stage:

	1	2	3	4	5	6	7	8	9
s_0	F	F	F	F	F	F			
s_1		F	F	F	F	F	F		
s_2			F	F	F	F	F	F	
s_3				F	F	F	F	F	F

stage 3: idle slots 1–3, warmup

At least $D_{pp} - 1 = 3$ warmup slots are idle on the last stage before steady state — and a matching drain effect appears at the end.

DeepSeek-V3 replaces this one-direction baseline with DualPipe: forward, input-gradient, and weight-gradient chunks injected from both ends so otherwise-idle stage time can overlap eligible communication. [REPORTED: DeepSeek-V3 technical report]

MoE axes: expert parallelism and placement

Expert parallelism: routing expands rows before it groups them

DeepSeek-V3 has 256 routed experts, activates 8 per token, and trains with 64-way expert parallelism across eight nodes. [REPORTED: DeepSeek-V3 technical report]

Toy fixture: 4 tokens, top-2 routing over 3 experts $\Rightarrow$ 8 token-expert rows. If $w_{1,0} = 0.55$ and $w_{1,2} = 0.45$,

$$y_{t_1} = 0.55 f_0(x_{t_1}) + 0.45 f_2(x_{t_1}).$$

The combine invariant: expert outputs must return to the *originating* token position, weighted exactly as dispatch recorded them. A fast dispatch that drops or misroutes a weight is simply wrong.

Placement and load balance are a separate ownership layer

Routing selects a *logical* expert; a placement map $\Phi_{\text{place}} \subseteq \mathcal{R} \times \mathcal{E}$ decides which *physical* rank(s) own it — without touching the token id, route slot, expert id, or combine weight.

Placement and load balance are a separate ownership layer

Routing selects a *logical* expert; a placement map $\Phi_{\text{place}} \subseteq \mathcal{R} \times \mathcal{E}$ decides which *physical* rank(s) own it — without touching the token id, route slot, expert id, or combine weight.

Histogram (40, 38, 15, 12, 8, 7) over a naive 4-rank placement gives loads (78, 27, 8, 7): legal, but rank 0 is the candidate straggler.

Placement and load balance are a separate ownership layer

Routing selects a *logical* expert; a placement map $\Phi_{\text{place}} \subseteq \mathcal{R} \times \mathcal{E}$ decides which *physical* rank(s) own it — without touching the token id, route slot, expert id, or combine weight.

Histogram (40, 38, 15, 12, 8, 7) over a naive 4-rank placement gives loads (78, 27, 8, 7): legal, but rank 0 is the candidate straggler.

A redundant expert copy trades memory for locality — and only helps if the runtime can still preserve combine semantics when two ranks own the same logical expert. [INSPECTED: EPLB `ep1b.py`]

Sequence axis and composition

SP shards activations; CP persists context chunks

Sequence parallelism (SP)	Context parallelism (CP)
shards selected activations inside a TP plan	assigns persistent sequence chunks to a CP group
no new full-context ownership axis	remote queries need remote mask-eligible KV

Query 6 on rank 1 still needs keys 1–4 from rank 0. Splitting context changes ownership; it does not make remote context disappear — softmax must normalize over the same mask-eligible keys as the unsharded computation.

From concept to code: one routed row, released end to end

REQUEST-TRACE/SCALED: row ρ_7 , slot 0, logical expert 6, weight 0.65. EPLB resolves a physical owner; DeepEP moves the row.

```
from DeepEP/deep_ep/buffers/elastic.py
```

```
def dispatch(self, x, topk_idx=None, topk_weights=None, ...):  
    """Dispatch tokens to different ranks."""  
    ...  
def combine(self, x, handle, topk_weights=None, ...):  
    """Combine (reduce) tokens ... back to their original ranks."""  
    ...
```


From concept to code: one routed row, released end to end

REQUEST-TRACE/SCALED: row ρ_7 , slot 0, logical expert 6, weight 0.65. EPLB resolves a physical owner; DeepEP moves the row.

```
from DeepEP/deep_ep/buffers/elastic.py
```

```
def dispatch(self, x, topk_idx=None, topk_weights=None, ...):  
    """Dispatch tokens to different ranks."""  
    ...  
def combine(self, x, handle, topk_weights=None, ...):  
    """Combine (reduce) tokens ... back to their original ranks."""  
    ...
```

`dispatch` carries the row to its physical expert owner under a placement-map version; `combine` restores the weighted result to ρ_7 using the same handle. Neither call chooses the route. [INSPECTED: DeepEP `deep_ep/buffers/elastic.py`]

Composition fails at edges, not inside one axis

Failure mode	Boundary
Collective mismatch	group/primitive/shape disagreement mid-sequence

Composition fails at edges, not inside one axis

Failure mode	Boundary
Collective mismatch	group/primitive/shape disagreement mid-sequence
Hot expert straggler	dispatch, compute, or combine boundary

Composition fails at edges, not inside one axis

Failure mode	Boundary
Collective mismatch	group/primitive/shape disagreement mid-sequence
Hot expert straggler	dispatch, compute, or combine boundary
Cache pressure	serving admission or decode

Composition fails at edges, not inside one axis

Failure mode	Boundary
Collective mismatch	group/primitive/shape disagreement mid-sequence
Hot expert straggler	dispatch, compute, or combine boundary
Cache pressure	serving admission or decode
Recovery gap	sharded state with no reconstructible owner

Composition fails at edges, not inside one axis

Failure mode	Boundary
Collective mismatch	group/primitive/shape disagreement mid-sequence
Hot expert straggler	dispatch, compute, or combine boundary
Cache pressure	serving admission or decode
Recovery gap	sharded state with no reconstructible owner

A plan can be locally valid on every axis and still deadlock at a cross-axis edge. The reviewed four-H200 composition rows found exactly this shape of failure: correct pieces, one broken handoff. [OBSERVED: four-H200 composition and DeepEP/EPLB/DualPipe replay]

**Training vs. serving: same
vocabulary, different owners**

The tuple changes what it owns, not its shape

$$\Gamma_{\text{train}} = (D_{\text{dp}}, D_{\text{tp}}, D_{\text{pp}}, D_{\text{ep}}, D_{\text{sp}}, Z, \mathcal{G}, \sigma)$$

$$\Gamma_{\text{serve}} = (O_{\text{req}}, D_{\text{tp}}, D_{\text{pp}}, D_{\text{ep}}, O_{\text{cache}}, Z=\emptyset, \mathcal{G}, \sigma)$$

- Training's critical path is step time: forward, backward, gradient sync, optimizer update.
- Serving's critical path is request latency: admission, prefill, KV-cache blocks, decode, response.

DeepSeek's disclosed system instantiates this by phase: EP32/DP32 for prefill, EP144/DP144 for decode — one reported online configuration, not a default implied by the model. [REPORTED:

DeepSeek-V3/R1 inference system overview]

Concept-to-code map for the chapter

- **logical routing** → `DeepSeek-V3/inference/model.py`
- **expert placement** → `EPLB/eplb.py`
- **expert dispatch and combine** → `DeepEP/deep_ep/buffers/elastic.py`
- **grouped expert compute** → `DeepGEMM/deep_gemm/__init__.py`
- **enclosing pipeline schedule** → `DualPipe/dualpipe/dualpipe.py`
- **checkpoint persistence** → `3FS/README.md`

Name the object, the owner, the consumer, and the boundary — before you trust the number.

Next: DeepEP, EPLB, DualPipe, and 3FS each open one of these boundaries in full, source-grounded depth.